

Inteligência Aumentada

CAPÍTULO 39

EVALS — A ENGENHARIA DE MEDIR IA

“Trocar prompt ‘porque ficou melhor’ sem golden set é torcida, não decisão. Eval é o que separa engenharia de fé.”

□ **Invariante-mãe: Invariante 7 — Termômetro** — “IA sem eval é fé, não engenharia.” Invariantes secundários: **Inv. 1** (Plausibilidade — eval comportamental detecta o que o modelo finge saber), **Inv. 3** (Camada Dupla — golden set é padrão durável; ferramenta de eval é número volátil), **Inv. 8** (Responsabilidade Indelegável — eval em CI é o que sustenta accountability técnica). Framework derivado: **F8 — EVAL-PIRÂMIDE**.

39.1 — O CONCEITO INTUITIVO

Há uma cena que se repete em times que adotaram IA generativa em produção sem ter passado pela disciplina de medi-la. O Product Manager pergunta “como está a qualidade do nosso copiloto?”, e a resposta vem em três versões. A primeira é “está boa, os

usuários têm gostado”, baseada em algumas conversas no Slack e em duas reuniões com clientes. A segunda é “passou na nossa bateria de testes da semana”, baseada em vinte casos rodados manualmente por um engenheiro. A terceira é “o LLM que avalia diz que melhorou”, baseada em um juiz que ninguém calibrou contra humano. Nenhuma das três é eval, e essa confusão custa caro porque mascara regressão até ela virar incidente.

Eval, em sentido técnico preciso, é a disciplina de medir sistematicamente a qualidade de saídas de IA contra critérios definidos, com dados representativos, em ciclos que rodam tanto em desenvolvimento quanto em produção, com critério de bloqueio explícito e auditoria do que cada release entregou. Não é teste manual, não é vibe, não é demonstração de cliente, não é benchmark público. É a infraestrutura cognitiva que transforma operação de IA em disciplina engenheirável.

A diferença entre quem tem eval real e quem não tem aparece em dois lugares. No deploy, porque quem tem eval consegue dizer “rodou contra 200 casos do golden set e melhorou em 92% deles, regrediu em 3% que estamos investigando, manteve em 5%”. Quem não tem só consegue dizer “ficou melhor”. No incidente, porque quem tem eval consegue rastrear a regressão até a mudança específica de prompt, modelo ou tool que a causou. Quem não tem entra em pânico, tenta voltar a uma versão “que parecia boa”, e descobre que perdeu o registro do que era.

A boa notícia é que eval não exige investimento absurdo. A pirâmide canônica desta disciplina, que apresentaremos formalmente como **Framework F8 EVAL-PIRÂMIDE**, tem base barata e topo caro, e a maioria dos times consegue valor real começando pela base. A má notícia é que a maior parte das organizações ainda não começou, e está tomando decisão de modelo, de prompt e de fornecedor sem instrumento para medir o que efetivamente entrega.

39.2 — ANALOGIA: O LABORATÓRIO CLÍNICO QUE NUNCA FECHA

Pense numa rede hospitalar séria, com cobertura nacional, plantão 24 horas, complexidade real. Imagine que essa rede operasse sem laboratório clínico próprio, sem exame de sangue regular, sem protocolo de checagem cruzada entre diagnósticos. A rede ainda atenderia pacientes — médicos competentes conseguem chegar em boas conclusões com anamnese, exame físico e bom senso. Mas quando algo desse errado, ninguém saberia identificar a causa. Quando um protocolo precisasse ser ajustado, a decisão seria por intuição. Quando o resultado de um departamento começasse a cair, a regressão seria descoberta tarde, quando já tivesse virado óbito.

Nenhuma rede hospitalar séria opera assim. Existe laboratório, existe protocolo de exame regular para cada classe de paciente, existe controle de qualidade dos próprios exames (sim, o laboratório precisa medir a si mesmo), existe trilha de auditoria por amostra. O laboratório não substitui o médico, e nem entrega o diagnóstico final. O laboratório dá ao médico o termômetro sem o qual a decisão fica no escuro.

Em IA, eval é o laboratório clínico do sistema. Não substitui o engenheiro, não decide pelo Product Manager, não escolhe o modelo. Dá ao time o termômetro com que se decide, se itera, se governa. E exige a mesma disciplina de qualidade que o laboratório clínico exige — golden set bem construído é o equivalente a amostras de controle calibradas, LLM-as-judge sem calibração é o equivalente a exame sem padrão de referência, regressão por subgrupo é o equivalente a olhar apenas a média sem ver o que acon-

teceu com a UTI pediátrica. Quem opera IA sem laboratório opera no escuro, e descobre as falhas pelo paciente — quer dizer, pelo cliente — que reclama tarde demais.

39.3 — EXPLICAÇÃO TÉCNICA

39.3.1 — Os dois eixos: offline × online

Eval é organizado em dois grandes eixos.

Eval offline acontece antes do deploy, sobre dataset controlado, com gabarito conhecido. É o que roda em pull request, em CI/CD, em release candidate. Sua função é detectar regressão antes que ela chegue ao usuário. Inclui golden set, adversarial set, drift set, testes determinísticos, LLM-as-judge calibrado, rubrica humana.

Eval online acontece em produção, sobre tráfego real, sem gabarito predefinido. Sua função é detectar problemas que escaparam ao offline e mapear lacunas do golden set. Inclui sampling de produção para rotulagem contínua, feedback de usuário (thumbs up/down, correção, abandono), monitoramento de drift de distribuição, monitoramento de custo e latência, monitoramento de segurança (jailbreak, prompt injection, conteúdo proibido).

Os dois se alimentam mutuamente em ciclo. Falha em produção que não foi capturada offline indica buraco no golden set, que entra no próximo ciclo de eval offline. Caso de teste novo que aparece no adversarial set é incluído no monitoramento de produção. Essa retroalimentação é o que mantém o laboratório vivo.

39.3.2 — A pirâmide canônica (Framework F8)

F8 — EVAL-PIRÂMIDE. *Hierarquia de evals por criticidade e custo. Três camadas verticais e uma transversal.*

Base — testes determinísticos. Schema validation (a saída obedece ao formato esperado), exact match (campos críticos batem com gabarito), regex (padrões obrigatórios estão presentes), validações estruturais (JSON parseável, citação no formato exigido). Cobertura: 100% das chamadas em CI. Custo: muito baixo. Detecta a maior parte das regressões grosseiras (formato quebrado, campo faltante, alucinação estrutural) antes que cheguem a camadas mais caras.

Meio — golden set com LLM-as-judge calibrado. Conjunto fixo de casos com gabarito de qualidade, julgado por LLM seguindo rubrica explícita, com calibração regular contra humano sênior. Cobertura: 30% a 80% das chamadas conforme criticidade. Custo: médio. Detecta regressão de qualidade que escapa da base e cobre casos de geração aberta onde exact match não funciona.

Topo — rubrica humana especialista. Revisão de amostra por especialista do domínio em release crítico, em auditoria mensal ou trimestral, em incidente sob investigação. Cobertura: 5% a 15% da carga, por amostra. Custo: alto. Detecta nuances que LLM-as-judge não capta e calibra o juiz LLM contra o critério humano.

Transversal — adversarial e segurança. Casos que sabidamente quebram o sistema (jailbreak, prompt injection, conteúdo proibido, sycophancy, viés), mantidos em conjunto separado, rodados a cada release crítico e em cadência mensal. Não substitui a pirâmide vertical; complementa.

A regra de bolso é construir a base primeiro, depois o meio, depois o topo. Quase todo time tem condição de subir a base em uma semana de trabalho. O meio leva entre uma e quatro semanas para a primeira versão sólida. O topo entra quando o produto cresce e a auditoria humana vira parte do contrato com o usuário ou do regime regulatório.

39.3.3 — Os 6 tipos canônicos de eval

Tipo	O que mede	Quando funciona	Armadilha clássica
De- ter- mi- nís- tica	Estrutura, campo crítico, padrão obrigatório	Saídas estruturadas (JSON, classificação, extração)	Saídas abertas; precisa combinar com outros tipos
Por simi- lari- dade	Embedding, BLEU, ROUGE, BERTScore	Geração aberta com gabarito de “resposta canônica”	Mede forma, não semântica; texto certo com paráfrase diferente pode pontuar mal
Ru- brica hu- mana	Critério de qualidade definido por especialista	Domínio específico, casos críticos	Lento, caro, baixo throughput; serve só no topo
LLM- as- judge	Rubrica explícita aplicada por outro LLM	Cobertura de volume com critério padronizado	Viés de auto-validação se juiz = gerador; precisa calibração contra humano
Com- por- ta- men- tal	Resposta a casos de segurança, viés, jailbreak	Compliance, alignment, governança	Conjunto fica desatualizado se não for renovado
Eco- nô- mica	Custo por resolução, latência, re-tentativa	Decisão de roteamento, otimização de stack	Ignorar pode mascarar regressão de qualidade que veio com troca de modelo “mais barato”

39.3.4 — Golden set: o ativo central da operação de eval

O golden set é o conjunto fixo de casos com gabarito que serve de referência estável para detectar regressão. É o ativo mais importante de qualquer operação madura de IA, e também o mais negligenciado.

Construção. Comece com 30 a 50 casos representativos da carga real (não de casos teóricos), cobrindo as classes principais de uso. Para cada caso, defina o gabarito junto com especialista do domínio — não é trivial decidir o que é “resposta certa” para uma pergunta aberta, e a definição precisa ser explícita. Documente o porquê de cada caso estar no golden, porque sem isso o conjunto vira manutenção opaca em seis meses. Mantenha proporção entre classes alinhada à distribuição real de produção.

Tamanho mínimo. Não há número universal. A regra prática é: 30 casos por classe crítica é o mínimo para começar a detectar regressão significativa, 100 a 200 por classe é confortável para a maioria dos times de tamanho médio, 500+ aparece em times maduros com cobertura larga. O golden set cresce com a operação, e o que importa é ser representativo e versionado, não enorme.

Manutenção. O golden set precisa ser revisado periodicamente. Casos que perderam relevância são aposentados. Casos novos que aparecem em produção (especialmente os que falharam) são incluídos. Distribuição é recalibrada se a carga de produção mudou. Sem manutenção, o golden vira folclore que não bate com a realidade do sistema.

Versionamento. Cada release que muda o golden set documenta a mudança. Cada decisão de eval que usa o golden registra a versão. Sem versionamento, “passou no eval” perde significado porque ninguém sabe contra o que passou.

39.3.5 — LLM-as-judge: quando confiar, quando duvidar

Usar um LLM para julgar a saída de outro LLM é técnica poderosa de escala, e é também uma das fontes mais comuns de eval enganoso. Vale entender em que condições funciona e em que condições produz ilusão.

Funciona quando: a rubrica é explícita e separa claramente “bom” de “ruim”; o juiz é modelo diferente do gerador (mitiga viés de auto-validação); a rubrica foi calibrada contra avaliadores humanos seniores em pelo menos 30 a 50 casos, com correlação alvo acima de 0,7; o juiz é re-calibrado mensalmente para detectar drift; ensemble de juizes é usado em decisões críticas; a saída do juiz inclui justificativa estruturada para amostragem auditável.

Não funciona quando: a rubrica é vaga (algo como “diga se ficou bom”); o juiz é o próprio gerador ou da mesma família (auto-validação inflada); não houve calibração contra humano; o juiz é tratado como verdade absoluta em decisão de produção sem cross-check; o adversarial set não inclui casos em que o juiz sabidamente erra.

Quadrante de decisão. Eixo X: especificidade da rubrica (vaga → muito específica). Eixo Y: risco da decisão (baixo → alto). Quadrante de baixo-esquerda (rubrica vaga, baixo risco): aceitar LLM-as-judge com revisão amostral. Baixo-direita (rubrica vaga, alto risco): exigir humano. Alto-esquerda (rubrica específica, baixo risco): LLM-as-judge isolado serve. Alto-direita (rubrica específica, alto risco): LLM-as-judge calibrado, com ensemble e auditoria humana de amostra.

39.3.6 — Métricas por tipo de tarefa

Não existe métrica universal. Cada tipo de tarefa tem métricas primária e secundária:

Tipo de tarefa	Métrica primária	Métricas secundárias
Classificação	F1 ponderada por classe	Precision/recall por classe, matriz de confusão
Extração estruturada	F1 por campo	Cobertura (% de chamadas com extração válida)
Geração aberta	Rubrica humana ou LLM-as-judge calibrado	Faithfulness (não inventou), relevância, completude, estilo
Resposta ancorada (RAG)	Faithfulness + answer relevance	Context precision, recall do retriever, citação correta
Agente	Taxa de sucesso de tarefa end-to-end	Passos, custo, regressão por subtarefa
Segurança	Taxa de rejeição correta	Over-refusal (recusas indevidas), taxa de jailbreak bem-sucedido
Robustez	Variância em N execuções	Consistência semântica
Custo/latência	CPR (cost per resolution)	CAA (custo agregado por agente), p50/p95 de latência

Regra dura sobre acurácia agregada. Acurácia agregada esconde regressão por subgrupo. Toda métrica de qualidade deve ter cortes — por classe, por segmento, por persona, por idioma, por regi-

ão, por canal. Times que olham só a média perdem regressão silenciosa que aparece em produção como reclamação concentrada de uma classe específica.

39.3.7 — Regressão em CI/CD: o gate que separa engenharia de fé

Toda mudança que afeta o comportamento do sistema — prompt, modelo, tool, system prompt, dataset de RAG, política de classificação — deve passar por eval automatizado antes de chegar à produção. A arquitetura canônica:

1. **Branch.** Engenheiro abre branch com a mudança proposta.
2. **Eval offline em CI.** Pipeline roda golden set + adversarial set automaticamente.
3. **Scorecard gerado.** Comparação automática com baseline da main, com delta por métrica e por subgrupo.
4. **Gate de bloqueio.** Política explícita do que bloqueia merge. Tipicamente: regressão acima de X% em métrica primária bloqueia automaticamente; regressão entre Y% e X% gera alerta para revisão humana; melhoria ou estabilidade permite merge.
5. **Code review.** Revisor olha código E scorecard. Mudança que melhora métrica mas piora subgrupo crítico não passa.
6. **Merge.** Após aprovação, mudança vai para staging.
7. **Canário em produção.** Deploy progressivo (1% → 10% → 50% → 100%) com eval online medindo qualidade real.
8. **Promoção ou rollback.** Critério explícito de promoção (scorecard verde em N horas) ou de rollback (regressão detectada).

Changelog de prompt é obrigatório a cada release. Documenta o que mudou, o motivo, o delta de scorecard, o subgrupo mais afetado. Sem changelog, troubleshooting de regressão em três meses vira arqueologia.

39.3.8 — Eval em produção: o ciclo que mantém o laboratório vivo

O eval offline detecta o que sabe procurar. O eval em produção descobre o que ninguém esperava. A arquitetura mínima:

Telemetria. Cada chamada loga input completo (sanitizado de PII conforme política de LGPD), output completo, latência por passo, tokens in/out, custo computado, versão de prompt, versão de tool, modelo usado, ID de sessão, ID de usuário (anonimizado conforme política). Sem telemetria, eval em produção não existe.

Sampling para rotulagem. Subconjunto representativo da carga é separado para rotulagem humana contínua. Tipicamente 0,5% a 2% do volume, distribuído por classe. Esses casos viram a próxima geração do golden set.

Feedback de usuário. Thumbs up/down, correção explícita, abandono, retentativa, escalonamento para humano. Cada sinal é proxy imperfeito de qualidade, mas a combinação informa onde olhar.

Loop fechado. Casos que falharam em produção e foram rotulados entram no próximo ciclo de eval offline. Casos novos no adversarial vêm de produção, não da cabeça do time. Sem esse loop, o golden set congela e o eval vira ritual.

39.3.9 — Os 7 anti-padrões mais frequentes (com antídoto)

Anti-padrão	Por que mata	Antídoto
Vibe-eval (“rodei 5 casos e ficou bom”)	Ruído pessoal vira norma; viés do testador inflado	Golden set fixo, versionado, mínimo 30 casos/classe
Métrica única agregada (“acurácia subiu 2%”)	Esconde regressão por subgrupo crítico	Cortes por classe, segmento, persona
LLM-as-judge sem calibração (“o juiz disse que melhorou”)	Auto-validação enviesada; juiz e gerador da mesma família correlacionam erros	Juiz ≠ gerador, calibração contra humano em 30+ casos, ensemble em decisões críticas
“Passou no MMLU”	Benchmark público não reflete a carga da empresa	Golden set próprio + adversarial sobre carga real
“Vamos medir depois”	Depois nunca chega; quando precisa, vira incidente	Gate de CI desde o primeiro deploy, mesmo que mínimo
“Eval é caro”	Mas regressão silenciosa é mais cara, e detectada tarde	Pirâmide com base barata; topo só onde justifica
Happy path only	Adversarial fica fora; sy-cophancy, jailbreak e viés passam	Conjunto adversarial separado, renovado por trimestre

39.4 — DIAGRAMAS

□ Diagrama 39.1 — A Pirâmide de Evals (F8)

(SVG a produzir: `imagens/L1-C39-img-01-piramide-evals.svg`)

Pirâmide vertical com três camadas. Base larga em creme, rotulada “Determinístico – 100% das chamadas, custo trivial”. Meio em laranja, rotulado “Golden set + LLM-as-judge calibrado – 30-80%, custo médio”. Topo em navy escuro, rotulado “Humano especialista – 5-15%, custo alto”. À direita, faixa transversal cinza com “Adversarial e Segurança”. Frase âncora ao pé: “construa a base, depois o meio, depois o topo”.

□ Diagrama 39.2 — Fluxo de regressão em CI

(SVG a produzir: `imagens/L1-C39-img-02-fluxo-regressao.svg`)

Cadeia horizontal: Branch → Eval offline (golden + adversarial) → Scorecard → Gate de bloqueio (decisão sim/não) → Code review → Merge → Canário 1% → Eval online → Promoção 100% ou Rollback. Setas. Caixinhas com regra de bloqueio explícita em cada nó de decisão.

□ **Diagrama 39.3 — Quadrante LLM-as-judge: quando confiar / quando duvidar**

(SVG  a produzir:

`imagens/L1-C39-img-03-quadrante-llm-as-judge.svg`)

Matriz 2×2. Eixo X horizontal: “especificidade da rubrica” (vaga ↔ muito específica). Eixo Y vertical: “risco da decisão” (baixo ↔ alto). Quatro quadrantes com decisão clara em cada um.

39.5 — EXEMPLO MEMORÁVEL: A CONSULTORIA QUE QUASE QUEBROU PELA AUSÊNCIA DE GOLDEN SET

△ **Cenário ilustrativo** — composto a partir de padrões observados em consultorias estratégicas brasileiras durante adoção de IA generativa; números são realistas mas não identificam empresa específica.

Atlas Strategy, consultoria brasileira de estratégia com cerca de 120 consultores, em 2025. Implementou um agente de redação de relatórios para clientes corporativos, com o objetivo de reduzir o tempo médio de produção de relatório mensal de 18 horas para algo entre 6 e 8 horas por consultor. O sistema integrava Claude

Sonnet para a redação, Skills proprietárias com voz autoral da Atlas, Projects por cliente com histórico, e RAG sobre base de cases anteriores.

Nas primeiras quatro semanas, o entusiasmo foi alto. Consultores entregavam mais relatórios em menos tempo. Sócios viam ganho de margem. A diretoria da Atlas considerou o projeto sucesso e direcionou investimento para expandir. Não havia golden set, não havia LLM-as-judge calibrado, não havia regressão em CI. A validação se resumia a três sócios lendo quatro relatórios na sexta-feira, e dando sinal verde se “tivessem soado certo”.

Na sétima semana, um cliente do setor industrial recebeu um relatório com três parágrafos que continham números trocados. As fórmulas eram corretas. Os valores de entrada estavam errados, copiados de outro caso do trimestre anterior. O cliente pegou o erro porque era ele quem produzia os números originais. Mandou e-mail. A sócia-fundadora abriu o e-mail, leu, e em quinze minutos sabia que tinha problema sério. Em dois dias, mapearam outros três relatórios entregues no mês com erros numéricos similares. Em uma semana, dois clientes adicionais reportaram inconsistências.

A investigação revelou o que sempre revela: duas mudanças de prompt feitas em sequência por engenheiros diferentes, ambas com boa intenção. A primeira para “tornar mais conciso”, a segunda para “soar mais executivo”. Cada mudança, isoladamente, melhorou a percepção de qualidade dos sócios na sexta de validação. Combinadas, corromperam o passo de extração numérica em silêncio. Não havia como saber que tinham corrompido, porque ninguém estava medindo *faithfulness* numérico contra um conjunto fixo de casos.

A Atlas fez três mudanças permanentes, todas alinhadas ao Invariante 7. Primeira, construiu golden set de 80 relatórios reais com gabarito de números e tese, anotado pelos três sócios em sessões dedicadas. Toda mudança de prompt passa a rodar contra esse conjunto antes de merge. Segunda, implementou LLM-as-judge com rubrica explícita para faithfulness numérico, calibrado contra os três sócios em 30 itens de calibração, com correlação alvo acima de 0,8. Terceira, criou scorecard versionado por release de prompt, com bloqueio automático se faithfulness numérico cair acima de 1 ponto contra o baseline.

Em seis meses, regressões silenciosas zeraram. O tempo de revisão humana por relatório caiu pela metade, porque o gate filtrava o ruído antes do humano ver. A Atlas perdeu dois clientes durante o incidente, recuperou um, e usa o caso internamente como exemplo do que custa não medir. A lição estrutural não está na escolha do modelo, da Skill ou do RAG. Está na ausência inicial do termômetro. *Eval não é luxo de big tech; é a diferença entre engenharia e fé. Quem não tem golden set não tem produto de IA, tem aposta documentada.*

▣ **PARA EXECUTIVOS** Se sua organização tem IA em produção tocando cliente, faça uma pergunta direta ao time técnico hoje: “qual o golden set do nosso sistema, e quando foi a última vez que rodamos regressão contra ele?”. Se a resposta envolver hesitação ou referência a “testes manuais”, você está em risco. *Eval é decisão de governança técnica, e exige investimento proporcional ao risco. Times maduros tratam golden set como ativo organizacional, com versionamento, dono nominal, e revisão trimestral.*

39.6 — QUANDO USAR / QUANDO EVITAR

Implantar eval formal (com golden set + CI + telemetria de produção) sempre que: - IA toca cliente externo - Saída entra em decisão financeira, jurídica, regulatória, clínica - Equipe muda prompt ou modelo mais de uma vez por mês - Existe SLA explícito ou implícito de qualidade - Caso regulado (LGPD com decisão automatizada, AI Act, ISO 42001) - Múltiplos engenheiros mexem no mesmo prompt

Subset mínimo viável (sem overhead completo) quando: - Piloto interno de baixo risco, com 1 usuário, por uma semana - Teste de ideia descartável que não vai à produção - Demo única para conselho ou prospect

Em todo caso intermediário, comece pela base da pirâmide. Suba camadas conforme o produto crescer.

39.7 — VANTAGENS E LIMITAÇÕES

Vantagem	Limitação
Transforma IA de fé em engenharia	Custo inicial alto: construir golden set é trabalho lento e demanda especialista do domínio
Permite migração de modelo e troca de prompt sem medo	Risco de overfit ao golden set, exigindo adversarial e produção rotulada como contrapesos
Cria scorecard auditável para governança e compliance	LLM-as-judge tem viés sistemático sem calibração; manutenção do juiz é trabalho recorrente
Detecta regressão silenciosa antes do usuário	Acurácia agregada engana; exige cortes por subgrupo que custam mais para reportar
Reduz MTTR (tempo de recuperação de incidente)	Cultura de eval demanda mudança organizacional; em times resistentes, vira teatro de compliance
Permite decisão de modelo em sua própria carga	Versão de modelo desatualizada no golden set distorce comparação; renovação periódica é obrigatória

39.8 — CONEXÕES COM OUTROS CAPÍTULOS

- □ Tokens como unidade de eval econômica → Cap 03
- □ RAG e métricas específicas (faithfulness, context precision) → Cap 06

- □ **Fine-tuning** como decisão que exige golden set sólido → [Cap 08](#)
 - □ **Agentes e métricas end-to-end de tarefa** → [Cap 12](#)
 - □ **AI Engineering** como disciplina que inclui eval → [Cap 14](#)
 - □ **Comparação de modelos por encaixe (eval na sua carga, não em benchmark público)** → [Cap 15](#)
 - □ **Economia de tokens e métricas de custo composto** → [Cap 36](#)
 - □ **LLMOps: eval em produção como pilar operacional** → [Cap 40](#)
 - □ **Alignment: eval comportamental para safety e sycophancy** → [Cap 41](#)
 - □ **Governança: eval como controle técnico canônico** → [Cap 42](#)
-

39.9 — RESUMO EXECUTIVO

Conceito	Síntese
Definição	Disciplina de medir sistematicamente qualidade de IA, com dados representativos, em ciclos automatizados, com critério de bloqueio explícito
Eixos	Offline (antes do deploy) × Online (em produção); ciclo de retroalimentação
Pirâmide F8	Base determinística (100%) → Meio com golden + LLM-as-judge (30-80%) → Topo humano (5-15%) + adversarial transversal
6 tipos	Determinística, similaridade, rubrica humana, LLM-as-judge, comportamental, econômica
Golden set	30-50 casos para começar; crescimento orgânico; versionamento obrigatório; manutenção trimestral
LLM-as-judge	Funciona com rubrica específica + juiz diferente do gerador + calibração contra humano (≥30 casos); duvidoso sem isso
Métricas	Variam por tipo de tarefa; acurácia agregada esconde regressão por subgrupo
CI	Gate explícito de bloqueio; scorecard com delta; canário em produção; rollback testado

Anti-padrões

Conceito	Síntese
	Vibe-eval, métrica única, juiz não calibrado, “passou no MMLU”, “depois”, “caro”, “happy path”

39.10 — CHECKLIST DO CAPÍTULO

- ☐ Distinguir, em uma frase, eval offline de eval online
- ☐ Descrever as três camadas da pirâmide F8 e a faixa de cobertura típica de cada
- ☐ Citar os 6 tipos canônicos de eval e quando cada um funciona
- ☐ Defender a regra “juiz ≠ gerador” para LLM-as-judge em mesa técnica
- ☐ Construir um golden set inicial de 30-50 casos do seu produto
- ☐ Escrever a rubrica de LLM-as-judge para um caso real, com 4-6 critérios objetivos
- ☐ Definir, no seu time, qual delta de métrica bloqueia merge e qual gera alerta
- ☐ Apresentar a Pirâmide de Evals em reunião executiva em até 5 minutos
- ☐

Identificar três cortes (subgrupos) que sua acurácia agregada está escondendo hoje

☐

Mapear, no seu produto, qual tipo de eval cobriria qual classe de falha

☐

Reconhecer, em três frases recentes do seu time, qual anti-padrão da seção 39.3.9 elas correspondem

39.11 — PERGUNTAS DE REVISÃO

1. Por que “ficou melhor” não é critério, e o que precisa entrar no lugar?
 2. Em que situação LLM-as-judge é viés institucionalizado, e como você detecta?
 3. Por que adversarial set importa mais que golden set no longo prazo?
 4. O que muda na decisão de migração de modelo quando há golden set, e o que muda quando não há?
 5. Por que a regra “juiz $\neq$ gerador” é estrutural, não preferência?
 6. Como o ciclo offline $\rightarrow$ online $\rightarrow$ offline mantém o eval vivo?
 7. Qual a relação entre o Invariante 7 (Termômetro) e o Invariante 8 (Responsabilidade Indelegável)?
 8. Por que acurácia agregada esconde regressão silenciosa, e que método combate?
 9. Em qual classe de produto faz sentido começar pelo topo da pirâmide?
-

39.12 — EXERCÍCIOS PRÁTICOS

Exercício 1 — Classificação de falhas reais. Liste 5 falhas reais (próprias ou do mercado) que sua operação enfrentou nos últimos 6 meses. Para cada uma, classifique: teria sido capturada por golden set? Por LLM-as-judge? Por adversarial? Por humano? Mapeie quais camadas da pirâmide faltam hoje na sua operação.

Exercício 2 — Pirâmide do seu produto. Desenhe a Pirâmide de Evals para o seu produto/operação. Atribua percentual de cobertura atual em cada camada. Identifique a camada mais frágil e proponha plano de remediação em 30 dias.

Exercício 3 — Rubrica de LLM-as-judge. Para um caso real do seu produto, escreva rubrica de LLM-as-judge com 4 a 6 critérios objetivos. Defina o que é resposta “5/5” e o que é “1/5” em cada critério. Calibre contra você mesmo em 10 casos. Em seguida, calibre contra um par. Compare. O que aprendeu?

Exercício 4 — Adversarial set inicial. Projete o adversarial set de 20 casos para o seu sistema. Inclua: pedidos potencialmente perigosos, pedidos com indução de sycophancy, casos com viés conhecido, jailbreak via instrução, prompt injection via dado de entrada. Rode contra o sistema atual e documente o resultado.

39.13 — PROJETO DO CAPÍTULO

Construir o Caderno de Evals v1 do seu produto/operação. Entregável em 8-12 páginas + opcional repositório:

1. Definição explícita do que é “resposta certa” para a sua classe de tarefa

2. Golden set inicial de 30-50 casos com gabarito anotado
3. Rubrica de LLM-as-judge calibrada contra humano em pelo menos 30 itens
4. Política de bloqueio em CI (qual delta bloqueia, qual alerta, qual passa)
5. Adversarial set de pelo menos 20 casos
6. Telemetria mínima de produção (o que logar, o que não logar)
7. Plano de revisão do golden set (trimestral) e do adversarial (mensal)
8. Dono nominal do Caderno de Evals, designado por escrito

Critério de qualidade do projeto. Outro engenheiro, sem contexto, consegue ler o caderno e rodar o eval contra uma nova mudança de prompt. Se precisar perguntar mais de três coisas, o caderno está incompleto.

39.14 — REFERÊNCIAS PRINCIPAIS

▣ **Frameworks e padrões** - OpenAI. *OpenAI Evals* (paper técnico e repositório, 2023-) - Anthropic. *inspect-ai* (framework de eval, 2024-) - Stanford CRFM. *HELM – Holistic Evaluation of Language Models* (Liang et al., 2022-) - BIG-bench: *Beyond the Imitation Game Benchmark* (Srivastava et al., 2022)

▣ **LLM-as-judge e suas armadilhas** - Zheng, L. et al. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena* (2023) - Liu, Y. et al. *G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment* (2023)

- **RAG e métricas específicas** - RAGAS: Automated Evaluation of Retrieval Augmented Generation (Es et al., 2023) - Faithfulness in NLG: Evaluation and Methods (vários, 2020-)
 - **Adversarial e segurança** - HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal (Mazeika et al., 2024) - JailbreakBench (Chao et al., 2024) - BBQ: A Hand-Built Bias Benchmark for Question Answering (Parrish et al., 2022)
 - **Sobre as dificuldades genuínas de avaliar LLMs** - Bowman, S. *Evaluating LLMs is a minefield* (palestra, 2023) - Karpathy, A. comentários públicos sobre limitações de benchmarks (2023-2024)
-

39.15 — VALIDAÇÃO UAU

#	Critério	Você consegue?
1	Clareza — Explicar para um diretor não-técnico em 90 segundos por que “ficou melhor” não é critério, usando a metáfora do laboratório clínico	☐
2	Profundidade — Defender em reunião técnica por que LLM-as-judge sem calibração contra humano é viés institucionalizado, citando o quadrante de decisão e a regra “juiz ≠ gerador”	☐
3	Aplicação — Iniciar a primeira versão do golden set do seu produto esta semana, com pelo menos 30 casos representativos	☐
4	Conexão — Articular como o Cap 39 amarra Inv. 7 (Termômetro), Cap 06 (RAG), Cap 40 (LLMOps), Cap 41 (Alignment) e Cap 42 (Governança)	☐
5	Curiosidade UAU — Está com vontade de entrar no Cap 40 para entender como esse	☐

#	Critério	Você consegue?
	eval roda em produção, com tracing, versionamento e rollback proporcionais	

5 de 5? Avance. Você passou a falar a língua que separa “usar IA” de “operar IA”. **3 ou 4?** Releia o Bloco 39.3.4 (Golden set) e 39.3.7 (Regressão em CI). É onde a maioria dos times tropeça por achar que “testar manualmente” basta. **Menos de 3?** Releia o capítulo completo, especialmente se sua operação já tem IA em produção sem golden set. O risco é proporcional ao volume.

▣ **Próximo:** Capítulo 40 — LLMOps: a operação de IA em produção

“Quem não tem golden set não tem produto de IA. Tem aposta documentada.”

CAPÍTULO 40

LLMOPS — A OPERAÇÃO DE IA EM PRODUÇÃO

“Autonomia sem observabilidade é loop com cartão de crédito. O que destrói orçamento de IA não é o preço do token, é a falta de instrumentação que faz cada incidente custar duas semanas para ser visto.”

□ **Invariante-mãe: Invariante 6 — Autonomia Proporcional** — “Só dê ao agente a autonomia que você consegue medir e desfazer.” Invariantes secundários: **Inv. 5** (Custo Composto — observabilidade é pré-requisito para controlar o termo composto), **Inv. 7** (Termômetro — eval em produção fecha o ciclo do eval offline), **Inv. 8** (Responsabilidade Indelegável — accountability técnica exige tracing e auditoria). Framework derivado: **F3 — AGENTE-PROP** e, em segundo nível, **F7 — COMPOSTO-3T**.

40.1 — O CONCEITO INTUITIVO

Há uma classe de equipe que adotou IA generativa em produção entre 2024 e 2026 e descobriu, no susto, que o trabalho mais difícil não estava em escolher o modelo. Estava em **operar**. Operar significa ter resposta para perguntas que aparecem sempre, sempre nas piores horas. *Quem está usando agente? Quanto custou o último mês, por feature? Por que aquele cliente recebeu uma resposta errada na quinta-feira? Em quanto tempo conseguimos voltar à versão de prompt de duas semanas atrás? Quem alterou esse system prompt? Quando? Por quê? Há logs?* Cada pergunta dessa é resposta a um pilar de LLMOps. Times que têm os pilares, respondem em minutos. Times que não têm, respondem em dias, e às vezes não respondem.

LLMOps é a disciplina que cobre essa operação. Não é MLOps clássico com nome novo, e essa distinção merece atenção. MLOps clássico foi construído sobre o ciclo “treinar modelo → versionar modelo → deployar modelo → monitorar drift de dados → retreinar”. O que se versiona é o modelo, o que se monitora é drift estatístico, o que se retreina é o modelo. LLMOps trabalha em outra arena. O modelo, na maioria dos casos, é externo, fornecido por Anthropic, OpenAI, Google. O que se versiona é **prompt, tool, system prompt, dataset de RAG, política de classificação**. O que se monitora inclui drift de distribuição, mas inclui também **alucinação, custo composto, latência variável, prompt injection, dependência de vendor**. O que se “retreina” é o prompt versionado e o golden set, não o modelo.

A confusão custa caro. Times que assumem “é igual a MLOps” descobrem que o ferramental e a cultura precisam ser adaptados. Times que assumem “não precisamos disso porque o modelo é ex-

terno” descobrem que o que opera não é o modelo, é o sistema em volta. Este capítulo é o que precisa ser ensinado para que o leitor pare de aprender LLMOps pelo incidente.

A boa notícia é que LLMOps tem **sete pilares canônicos**, definidos com clareza suficiente para virar checklist. A regra prática é construir do pilar 1 ao pilar 7 conforme o produto amadurece, e nunca subir nível de autonomia (F3) além do que os pilares conseguem sustentar.

40.2 — ANALOGIA: A SALA DE CONTROLE DE UMA USINA

Pense numa usina hidrelétrica de porte médio. Operar não é apenas ligar a turbina. Operar é manter, em qualquer instante, **visibilidade completa** do que está acontecendo, **registro inalterável** do que aconteceu, **capacidade de reagir** ao que está acontecendo agora, e **capacidade de reverter** a uma configuração conhecida-segura quando o operador suspeitar que algo está fora do padrão.

A sala de controle tem painéis de instrumentação que mostram vazão, pressão, temperatura por turbina, voltagem por gerador, frequência da rede. Tem botões físicos para reduzir produção, para desligar, para desviar fluxo. Tem registros automáticos do que cada operador tocou, com timestamp e identificação. Tem procedimento documentado para cada classe de evento, com gatilho explícito. Tem revezamento de turno com handover formal. Tem auditoria periódica do próprio painel — porque painel que não é auditado vira mentira tranquilizadora.

LLMOps é a sala de controle da operação de IA. Cada pilar dos sete cumpre função análoga a um sistema na usina. Tracing é a instrumentação. Versionamento é o registro inalterável. Deploy progressivo é a abertura controlada de fluxo. Rollback é o desvio de emergência. Custo é o medidor de combustível. Segurança operacional é o protocolo de incidente. Governança operacional é o handover formal e a auditoria. Quando os sete funcionam, a operação atravessa incidentes com cabeça fria e tempo de recuperação que cabe em horas, não semanas. Quando faltam, cada evento vira investigação arqueológica.

40.3 — EXPLICAÇÃO TÉCNICA — OS 7 PILARES

40.3.1 — Pilar 1: Tracing e observabilidade

O que cobre. Cada chamada de IA é registrada com input completo, output completo, latência total e por passo, tokens in/out por passo, custo computado, tools chamadas com argumentos e retornos, modelo usado, versão de prompt, versão de tool, ID de sessão e de usuário, status (sucesso/erro/timeout). Sem este pilar, não há LLMOps possível.

Modelo mental canônico: **span** × **trace** × **session**. Cada chamada individual ao modelo é um **span**. Uma sequência de spans relacionados (por exemplo, um turno de conversa com múltiplas chamadas ao modelo e a tools) é um **trace**. Uma sequência de traces no mesmo contexto de uso (por exemplo, uma sessão de chat com vários turnos) é uma **session**. A nomenclatura vem de Open-Telemetry, e padroniza a discussão entre times.

O que logar. Input completo (sanitizado de PII conforme política de LGPD); output completo; latência por passo; tokens de input e output; custo computado em tempo real; tool chamada com argumentos e retornos; modelo usado; versão de prompt; versão de tool; ID de sessão; ID de usuário (anonimizado conforme política); status. Acrescentar metadados de contexto da operação (feature, ambiente, região).

O que NÃO logar. PII desnecessária ao debug. Segredos. Conteúdo sob LGPD sem base legal explícita. Saúde, orientação sexual, dados financeiros íntimos. Times maduros têm classificador automático de PII no pipeline de logging.

OpenTelemetry para LLMs. A especificação OpenTelemetry GenAI semantic conventions, em evolução desde 2024, padroniza nomes de atributos para spans de modelo, tool e agente. Adotar o padrão (mesmo que parcial) reduz vendor lock-in de observability. Vale o esforço quando o stack está em mais de um vendor.

40.3.2 — Pilar 2: Versionamento do que muda

O que se versiona em LLMOps. Prompt do sistema, prompt da feature, definição de tool (schema, descrição, exemplos), dataset de eval, política de classificação. Cada um vira artefato com versão semântica, dono, changelog explícito.

Promptware como código. Tratar prompt como código: PR (pull request), revisão obrigatória por pelo menos um par, eval em CI antes do merge, scorecard comparativo gerado automaticamente, merge gatekeeping. Engenheiros experientes em desenvolvimento moderno reconhecem o padrão imediatamente. A diferença é que o “compilador” é o sistema de eval, e o “lint” é o LLM-as-judge contra rubrica.

Tool registry. Cada tool em produção tem entrada em registry com schema (input/output), descrição, exemplos, dono operacional, versão, eval específico. Tool que muda sem entrar no registry é o caminho mais rápido para regressão silenciosa em agentes.

Model registry interno. Tabela que mapeia “qual modelo, qual versão, qual fallback” por feature. Mudança aqui é decisão de arquitetura, não de engenheiro, e deve passar por governança (Inv. 8).

System prompt como artefato versionado, não string no código. Pode estar em banco, em config, em sistema dedicado (LangSmith, Langfuse, Braintrust, sistema próprio). O que importa é não ser concatenação inline no código da feature.

Changelog obrigatório. Cada release que mexe em prompt, tool ou modelo registra: o que mudou, motivo, delta de scorecard, subgrupo mais afetado, decisão de revisor. Sem changelog, troubleshooting em três meses vira arqueologia.

40.3.3 — Pilar 3: Deploy progressivo

Shadow mode. Versão nova roda em paralelo, sem servir resposta ao usuário, com input real de produção. As saídas das duas versões (atual e candidata) são comparadas offline. Útil para validar mudança de modelo ou de prompt em escala, sem risco para o usuário final.

Canário por percentual de tráfego. Versão nova recebe 1% do tráfego, depois 10%, 50%, 100% conforme métricas sustentam promoção. Cada gate tem critério explícito: scorecard verde por N horas, latência dentro da banda, custo dentro do envelope, sem incidente SEV-2.

Canário por segmento. Estratégia complementar: liberar primeiro para clientes free, depois beta, depois SMB, e por último Enterprise crítico. Nunca o inverso. A regra é “quem reclamaria mais alto recebe por último”.

Feature flag. Liberar por persona, conta, região, ambiente. Permite isolar testes A/B e desligar versão problemática em segundos sem mexer em código.

CrITÉrios de promoção entre gates. Documentados, automáticos quando possível. Tipicamente: métrica primária dentro de banda esperada, latência p95 dentro da banda, custo dentro do envelope, taxa de erro estável, sem incidente em N horas.

40.3.4 — Pilar 4: Rollback que funciona em produção

Botão único por feature. Reverter prompt, tool ou versão de modelo deve estar a um clique de distância. Quem precisa de pull request, build e deploy para rollback descobre, no incidente, que rollback assim demora mais que o incidente.

Estado anterior em hot standby. A versão anterior sempre está pronta para servir. Não está só “no histórico do Git”. Está provisionada, com cache aquecido, esperando ser ativada.

Teste mensal em staging. Rollback que nunca é testado, na hora do incidente, não funciona. Times maduros simulam rollback uma vez por mês em staging, com cronômetro, com runbook seguido. Documentam quanto tempo levou e o que falhou.

MTTR como métrica institucional. Mean Time To Recovery — tempo médio entre detecção e retorno ao estado bom — vira métrica de OKR técnico. Times maduros ficam abaixo de 15 minutos para SEV-1; abaixo de 1h para SEV-2.

40.3.5 — Pilar 5: Custos, quotas e circuit breakers

Atribuição de custo por feature, cliente, usuário. Sem atribuição, não há como decidir o que cortar quando o orçamento aperta. Custo agregado mensal é informação inútil para decisão; custo por feature mostra onde está o predador.

Alertas de orçamento. Limites diários, semanais e mensais por feature e por cliente. Acionados antes do CFO ver.

Circuit breaker por usuário e por sessão. Limite duro de chamadas ao modelo premium por janela de tempo. Protege contra runaway loop em agente e contra abuso de usuário malicioso. Bug que causa loop fica contido em dezena de centavos, não em dezenas de milhares de reais.

Quota por tier. Não permitir que feature crítica seja sufocada por feature secundária em surto. Reserva de capacidade no modelo premium.

Fórmula F7 COMPOSTO-3T como referência. A ferramenta de redução de custo é o framework de Custo Composto. Sem ele, o pilar 5 fica em “alarmar quando estoura”.

40.3.6 — Pilar 6: Segurança operacional

Prompt injection. Defesa em camadas. Sandbox de tool. Escape de input do usuário. Allowlist de tools por contexto. Classificação prévia de input suspeito. Auditoria de saída antes de retornar ao próximo modelo na cadeia. Em agentes que consomem documentos externos, prompt injection é vetor primário; assumir que existe é mais seguro que esperar que não.

Tool poisoning. Validar a saída de tool antes de devolver ao modelo. Tool que retorna conteúdo malicioso pode contaminar o agente. Sandbox por tipo de retorno + validação contra schema.

Data exfiltration. Política sobre o que o modelo pode mandar para fora via tool. Whitelist de domínios para chamadas externas. Auditoria periódica de logs de tools externas.

Kill switch testado. Capacidade de desligar uma tool, um agente, um modelo, ou uma feature inteira em segundos. Por escopo. Testado em simulado mensal. Dono nominal por escopo.

40.3.7 — Pilar 7: Governança operacional

Severidades adaptadas a IA. SEV-1 (impacto crítico em cliente, exposição de PII, perda de dados, erro grave em decisão automatizada). SEV-2 (degradação significativa, custos disparados, regressão de qualidade detectada). SEV-3 (problema isolado, baixo impacto). SEV-4 (irritação, sem perda).

Comunicação durante incidente. Canal dedicado. Atualizações em cadência fixa (a cada 15 minutos em SEV-1). Comunicação externa pré-escrita por classe de incidente. Comunicação interna estruturada por status (detectado, contido, investigando, mitigado, resolvido).

Postmorte sem culpa. Foco no processo, não na pessoa. Discussão estruturada: o que aconteceu, quando, como foi detectado, quanto tempo levou para conter, o que poderia ter sido feito diferente, quais mudanças vão evitar repetição. Documento publicado, lido por todos.

Trilha de auditoria. Retenção mínima 90 dias para casos não-críticos; 5 anos ou mais para decisão regulada (LGPD com decisão automatizada, ANPD, AI Act, ISO 42001).

40.4 — DIAGRAMAS

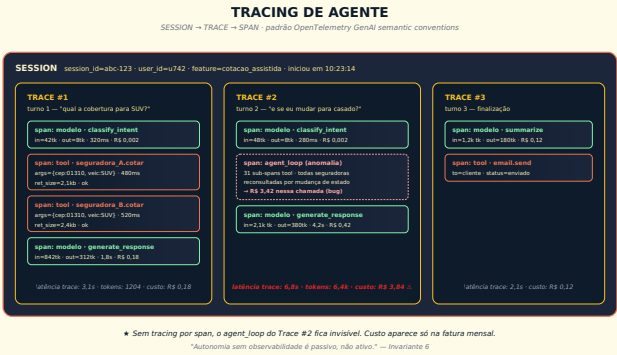
□ Diagrama 40.1 — Stack canônico de LLMOps



Stack canônico de LLMOps

Os 7 pilares operacionais alinhados por dono e por fluxo de telemetria. A pirâmide funcional vai do que rastrear (base) ao que governar (topo).

□ Diagrama 40.2 – Tracing de agente: sessão → trace → spans



Tracing de agente

Modelo mental canônico. Cada chamada individual é um `span`; conjunto relacionado é um `trace`; conjunto temporal é uma `session`. Padronização OpenTelemetry GenAI.

□ Diagrama 40.3 — Linha do tempo de incidente



Timeline de incidente

Do T0 (detecção) ao D+5 (postmortem publicado). Cada nó com pré-requisito explícito do pilar de LLMOps que o sustenta.

40.5 — RECORTES POR PERSONA

A pergunta “o que cada papel precisa saber de LLMOps?” tem respostas diferentes. A tabela abaixo separa por persona, com métrica-chave e pergunta certa a fazer ao fornecedor.

Dimensão	CTO	Arquiteto	Desenvolvedor
O que precisa saber	7 pilares no nível de decisão e custo; trade-offs build × buy; orçamento operacional; RACI por feature	Arquitetura do stack; integração com observability existente; padrão de versionamento; modelo de span/trace; pontos de extensão	Como instrumentar; como adicionar tracing custom; como criar PR de prompt; como rodar canário; como ler scorecard
O que NÃO precisa saber	Detalhe de implementação de span; biblioteca específica	Métricas executivas como CPR agregado por LOB	Política de incidente nível executivo; trilha regulatória de retenção
Métrica-chave	Custo total de IA / receita	Cobertura de tracing %; MTTR rollback; cobertura de versionamento;	Cobertura de tracing nas próprias features; tempo médio para identificar bug de IA

Dimensão	CTO	Arquiteto	Desenvolvedor
	ta; SLA de IA; SEV-1 e SEV-2 por tri- mes- tre	cobertura de testes de inci- dente	
Pergunta certa ao fornecedor	“Como atribui custo de IA por cliente final?”	“Vocês têm OpenTelemetry GenAI? Que versão?”	“Como o SDK me dá hook para tracing customizado e propagation de context?”

40.6 — EXEMPLO MEMORÁVEL: PÓLICE.IO E O LOOP COM CARTÃO DE CRÉDITO

△ **Cenário ilustrativo** — composto a partir de padrões observados em marketplaces e fintechs brasileiras durante a adoção de agentes integrados a múltiplos APIs por MCP em 2025-2026; números são realistas mas não identificam empresa específica.

Pólíce.io, marketplace brasileiro de seguros, em 2026, com cerca de oitenta colaboradores. Implementou um agente de cotação assistida via chat, integrado a sete seguradoras por MCP, com objetivo de reduzir o tempo de cotação de quinze minutos (humano) para menos de dois minutos (agente com humano revisando). Nas três primeiras semanas em produção, sucesso aparente. Conversões subiram dezessete por cento. NPS estável. O CFO comentou em reunião que a fatura de IA estava crescendo “como esperado”.

Na quarta semana, a fatura ultrapassou R\$ 142 mil. No mês zero do projeto, antes do agente, a fatura era R\$ 18 mil. A diferença foi atribuída pela equipe técnica a “uso maior do que esperado”. O CFO aceitou a explicação, embora inquieto. Na quinta semana, a fatura cruzou R\$ 200 mil projetados. O CTO da Pólíce.io começou a investigar. Não tinha tracing instrumentado. Não tinha atribuição por feature. Não tinha circuit breaker. Tinha logs do modelo (input/output básico) e a fatura agregada.

Levou onze dias para a equipe descobrir o que estava acontecendo. Um bug introduzido em PR aparentemente inofensivo na terceira semana — refatoração de um campo do formulário, sem nada que parecesse relevante para o agente — fazia com que, sempre que o usuário trocasse de estado civil no meio do fluxo, o agente reconsultasse as sete seguradoras integradas por MCP. Cada reconfiguração disparava chamadas ao Opus para reavaliar contexto. Em média, **trinta e uma chamadas redundantes ao modelo premium por cotação efetiva**. Multiplicado pelo volume crescente, a fatura escalou exponencialmente.

Quando finalmente instrumentaram tracing por span — duas semanas no telhado, urgência —, o problema apareceu em uma hora. Correção em duas. Custo do incidente: aproximadamente R\$ 96

mil em chamadas desperdiçadas (excedente da fatura no período), mais a credibilidade abalada da diretoria, mais duas semanas de engenharia desviadas para “salvar o orçamento”.

A Pólíce.io reescreveu o stack operacional em quatro semanas. Implementou tracing OpenTelemetry-style desde a primeira chamada de cada turno. Adicionou versionamento de tool com eval específico. Criou canário por percentual antes de qualquer promoção em produção. Estabeleceu circuit breaker por usuário com limite de cinco chamadas ao Opus por sessão. Alertas de custo por feature, por cliente, por usuário. Política de incidente SEV-1 com SLA de 15 minutos para detecção.

Em três meses, a fatura caiu para R\$ 47 mil mensais — com volume maior de cotações servidas que no período do incidente. A explicação não foi “ficamos mais rápidos no roteamento” ou “mudamos de modelo”. Foi: **observabilidade revelou onde o custo estava sendo desperdiçado**, e o time arrumou. O Cap 36 mostrou a fórmula; o Cap 40 mostra que sem o pilar 1 não dá para aplicar a fórmula.

A lição é dura. *Autonomia sem observabilidade é loop com cartão de crédito. O que destrói orçamento de IA não é o preço do token, é a falta de instrumentação que faz cada incidente custar duas semanas para ser visto.*

▣ **PARA EXECUTIVOS** Se a fatura mensal de IA da sua organização ultrapassou R\$ 30 mil e o time técnico não consegue responder “qual a feature mais cara?” em até 5 minutos, você está em risco. Atribuição de custo por feature é o primeiro item de auditoria de LLMOps em qualquer operação madura. Se faltar, peça plano de remediação em 30 dias.

40.7 — QUANDO USAR / QUANDO EVITAR

Implantar stack completo de LLMOps quando: - Há IA em produção tocando cliente - Há mais de um agente em produção - Há custo recorrente acima de R\$ 5 mil/mês em chamadas a LLMs - Há SLA explícito ou implícito de qualidade - Há requisito regulatório (LGPD com decisão automatizada, regulação setorial)

Subset mínimo viável (sem overhead completo) quando: - Piloto único interno, fora de produção, com 1 usuário - Demo única para conselho ou prospect - Teste de ideia descartável

Em todo caso intermediário, comece pelos pilares 1 (tracing), 4 (rollback) e 7 (governança). Os outros entram conforme escalar.

40.8 — VANTAGENS E LIMITAÇÕES

Vantagem	Limitação
Custo de IA cai 20-50% só com observabilidade bem feita	Custo inicial de instrumentação e padronização — tipicamente 4-12 semanas para o stack inicial
MTTR de incidentes cai dramaticamente — de dias para horas	Risco de over-engineering em produto early-stage
Audit trail vira ativo para compliance e disputa judicial	Requer cultura de versionamento que muitos times de IA não têm; demanda mudança organizacional
Permite migração de modelo sem terror	Vendor lock-in se observability vem de um único fornecedor proprietário; OpenTelemetry mitiga
Reduz dependência de “engenheiro herói” que conhece o sistema	Adiciona camada de processo que pode atrasar prototipagem
Habilita governança técnica (Inv. 8) com base auditável	Custo de armazenamento e compliance de logs cresce com volume

40.9 — CONEXÕES COM OUTROS CAPÍTULOS

- □ **Tokens como insumo da fórmula de custo** → [Cap 03](#)
- □ **Memória persistente e custo composto** → [Cap 07](#)
- □ **Agentes como classe operacional** → [Cap 12](#)
- □ **MCP como integração governável** → [Cap 13](#)

- □ **AI Engineering como disciplina inclusiva de LLMOps** → [Cap 14](#)
 - □ **Economia de tokens como aplicação do pilar 5** → [Cap 36](#)
 - □ **Segurança e mapa de risco operacional** → [Cap 37](#)
 - □ **Evals como insumo do pilar 1, 2 e 3** → [Cap 39](#)
 - □ **Alignment como insumo de segurança operacional** → [Cap 41](#)
 - □ **Governança corporativa que sustenta o pilar 7** → [Cap 42](#)
 - □ **Claude Code como caso de agente com Autonomia Proporcional aplicada** → [Cap 24 \(L2\)](#)
 - □ **Enterprise e governança em escala** → [Cap 30 \(L2\)](#)
-

40.10 — RESUMO EXECUTIVO

Pilar	Pergunta executiva	Indicador-chave
1 Tracing/observability	“Consigo reconstruir qualquer decisão da IA?”	% de spans com tracing completo
2 Versionamento	“Sei o que estava em produção em qualquer data?”	% de prompts/tools versionados
3 Deploy progressivo	“Posso testar com 1% antes de 100%?”	% de releases com canário
4 Rollback	“Quanto tempo do ‘algo errado’ ao estado anterior?”	MTTR de SEV-1
5 Custos/quotas	“Vejo agente loopando antes do CFO ver?”	Custo por feature, alerta antes de incidente
6 Segurança operacional	“Desligo uma tool perigosa em 30s?”	Tempo de kill switch testado
7 Governança operacional	“Quem responde quando der ruim?”	Auditoria de SEV-1 com postmortem público

40.11 — CHECKLIST DO CAPÍTULO

☐

Distinguir LLMOps de MLOps clássico em uma frase

☐

Listar os 7 pilares na ordem de impacto e a pergunta executiva de cada um

☐

Descrever o modelo mental span × trace × session

☐

Defender adoção de OpenTelemetry GenAI vs. vendor proprietário

☐

Identificar, no seu produto, o pilar mais frágil hoje

☐

Calcular o MTTR atual da sua operação e propor meta de 30 dias

☐

Apresentar a recortes por persona em reunião de tech leadership

☐

Reconhecer, em três incidentes recentes, qual pilar teria mitigado cada um

40.12 — PERGUNTAS DE REVISÃO

1. Por que LLMOps não é “MLOps com nome novo”, e o que muda na prática?
2. Em qual dos 7 pilares está o ROI mais imediato de implementar primeiro, e por quê?
3. Por que rollback “no Git” não é rollback de produção?

4. Como o pilar 1 sustenta a fórmula do Custo Composto (Cap 36)?
 5. Que diferença existe entre canário por percentual e canário por segmento, e quando usar cada um?
 6. Por que kill switch teórico é pior que não ter kill switch?
 7. Como o pilar 7 (governança operacional) conecta com o Invariante 8 (Responsabilidade Indelegável)?
 8. Em que situação OpenTelemetry GenAI é over-engineering, e em que situação é proteção contra lock-in?
 9. Como o pilar 5 (custos) prevê os anti-padrões do framework F7 COMPOSTO-3T?
-

40.13 — EXERCÍCIOS PRÁTICOS

Exercício 1 — Diagnóstico dos 7 pilares. Mapeie cada um dos 7 pilares no seu stack atual e atribua maturidade em escala 0-4 (0 inexistente, 1 declarado, 2 implementado, 3 auditado, 4 melhoria contínua). Identifique os dois pilares mais frágeis e proponha plano de 60 dias.

Exercício 2 — Trace ideal de uma sessão real. Para uma sessão real do seu produto, desenhe o trace ideal span por span. Inclua: chamadas ao modelo principal, chamadas a tools, chamadas a sub-agentes, latência, tokens, custo. Compare com o que seu sistema loga hoje. O que falta?

Exercício 3 — Playbook de rollback em 1 página. Escreva o playbook de rollback do seu prompt principal. Quem detecta, quem aciona, quanto tempo de cada passo, onde está o estado anterior, como confirmar que rollback funcionou. Limite: 1 página.

Exercício 4 — Cálculo do custo real do último mês. Use a fórmula do F7 COMPOSTO-3T (tokens × chamadas × redundância × tier) para reconstruir o custo do mês anterior. Identifique o componente que mais escalou. Proponha plano de redução de 25% em 60 dias.

40.14 — PROJETO DO CAPÍTULO

Construir o Caderno de LLMOps v1 do seu produto/operação. Entregável em 8-12 páginas, contendo:

1. Descrição dos 7 pilares aplicados ao próprio produto, com maturidade atual e maturidade-meta de 90 dias
2. Arquitetura de tracing (modelo span × trace × session aplicado)
3. Política de versionamento de prompt e tool
4. Runbook de rollback testado mensalmente
5. Runbook de incidente SEV-1 (detecção, contenção, comunicação, postmortem)
6. Atribuição de custo por feature, cliente, usuário
7. Dono nominal de cada pilar
8. Plano de revisão trimestral do Caderno

Critério de qualidade. Outro engenheiro, sem contexto, conseguir ler o caderno e responder, sem perguntar mais, qualquer das 7 perguntas executivas da seção 40.10 sobre o seu sistema.

40.15 — REFERÊNCIAS PRINCIPAIS

- **OpenTelemetry e padrões** - OpenTelemetry GenAI semantic conventions (2024-, em evolução) - *Distributed Tracing in Practice* (Parker, Spoonhower, Mace, 2020) — fundamento conceitual
 - **Frameworks e stacks LLMOps** - a16z. *The Emerging Architectures for LLM Applications* (Bornstein & Radovanovic, 2023) - Karpathy, A. *Software 3.0* (palestra, 2024) — visão de stack
 - **SRE e fundamento de operação** - Google. *Site Reliability Engineering Book* (Beyer et al., 2016) — capítulos sobre postmortem, MTTR, gestão de incidentes - Google. *The Site Reliability Workbook* (Beyer et al., 2018) — práticas operacionais
 - **Observability de LLMs (vendors)** - Documentação LangSmith, Langfuse, Helicone, Arize Phoenix, Datadog APM with LLM observability — padrões de mercado
 - **Segurança de LLMs em produção** - Greshake, K. et al. *Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection* (2023) - Liu, Y. et al. *Prompt Injection attack against LLM-integrated Applications* (2023) - Anthropic. *Responsible Scaling Policy* (2023, 2024)
 - **Alignment e operação** - Anthropic. *Constitutional AI* (Bai et al., 2022) — fundamentos de safety operacional
-

40.16 — VALIDAÇÃO UAU

#	Critério	Você consegue?
1	Clareza — Explicar a um sócio o que é tracing em IA usando a analogia da caixa-preta aeronáutica em 2 minutos	☐
2	Profundidade — Defender em reunião de arquitetura por que versionamento de prompt é tão crítico quanto versionamento de código	☐
3	Aplicação — Apontar, no seu produto, qual dos 7 pilares é o mais frágil hoje e o que faria nos próximos 14 dias	☐
4	Conexão — Articular como LLMops fecha o ciclo aberto por Cap 39 (Evals) e fundamenta Cap 42 (Governança), passando pela fórmula do Cap 36 (Custo Composto)	☐
5	Curiosidade UAU — Está com vontade de entrar no Cap 41 para entender como o próprio modelo é molda-	☐

#	Critério	Você consegue?
	do antes de chegar à sua operação	

5 de 5? Avance. Você passou de “usuário de IA” a “operador de IA”.
3 ou 4? Releia §40.3.1 (Tracing) e §40.3.5 (Custos). É onde a maior parte do investimento de LLMOps é desperdiçada por falta de prioridade correta. **Menos de 3?** Releitura completa, especialmente se você já tem agente em produção. O risco é proporcional ao tempo sem instrumentação.

▣ **Próximo:** Capítulo 41 — Alignment: como os modelos são moldados antes de chegarem a você

“O modelo é o motor. LLMOps é a sala de controle. Sem sala de controle, o motor decide o destino. Quem opera IA sem sala de controle não está operando — está sendo operado.”